Smart Laundry Management System

Project Report

```
┌──────────────────────────────────────────────────┐
│                                                  │
│        SMART LAUNDRY MANAGEMENT SYSTEM           │
│                                                  │
│            Academic Project Report               │
│                                                  │
│                                                  │
│        Submitted by: Muskan Sukhija              │
│        Reg Number: 24BAI10927                    │
│        Course: B.Tech                            │
│        Subject: Programming in Java (CSE2006)    │
│        Session: 2025 - 2026                      │
│                                                  │
│        Date: 30/3/2026                           │
│                                                  │
│                                                  │
│      VITyarthi - Build Your Own Project          │
│                                                  │
└──────────────────────────────────────────────────┘
```

1. Introduction

In today's fast-paced lifestyle, efficient management of daily tasks is essential, especially in shared living environments such as hostels and apartment complexes. Laundry management in these places is often handled manually using registers or informal methods, which leads to inefficiencies such as misplaced clothes, lack of order tracking, delays, and human errors.

The **Smart Laundry Management System** is a Java-based console application developed to address these challenges. It provides a structured and automated way to manage laundry services, enabling users to place orders, track their status, and maintain records digitally.

This system is specifically designed for **small-scale environments like hostels and apartments**, where a lightweight, easy-to-use solution is more practical than complex enterprise systems.

1.2 Problem Statement

Laundry services in hostels and small residential setups lack automation. The absence of a proper system results in:

- Manual record-keeping prone to errors
- Difficulty in tracking order status
- Inefficient handling of multiple users
- Lack of transparency for users
- No persistent storage of data

Hence, there is a need for a simple, reliable, and efficient system to manage laundry operations.

1.3 Objectives

The objectives of this project are:

- To design and develop an automated laundry management system
- To provide a user-friendly interface for placing and managing orders
- To implement dynamic pricing based on service type and cloth type
- To store and retrieve data using file handling
- To minimize manual errors and improve efficiency
- To provide a scalable system that can be enhanced in the future

1.4 Scope of the project

This system is designed specifically for:

- Hostels
- Small apartment complexes
- Small-scale laundry services

- Scope Limitations:

- No database integration (uses file handling)
- No multi-user authentication
- No online payment system
- Console-based interface

Despite these limitations, the system effectively fulfills its purpose in small environments.

2. Literature Review / Existing System

2.1 Existing Systems

The existing laundry system is mostly manual and involves maintaining records in notebooks.

2.2 Limitations of Existing Systems

- Time-consuming process
- High probability of human errors
- No proper tracking mechanism
- Risk of data loss
- Difficult to manage large number of orders

3. Proposed System

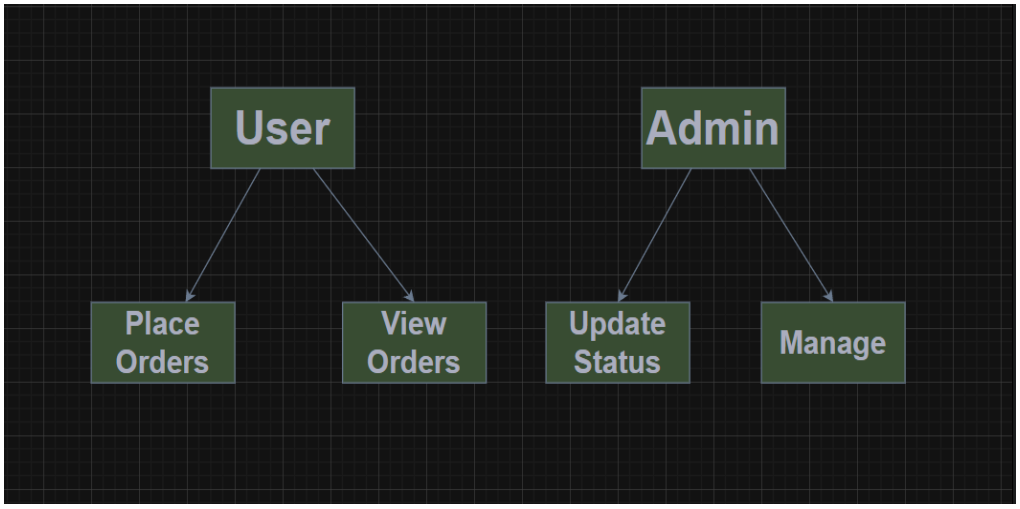
The proposed system is a **Smart Laundry Management System** that automates the process.

3.1 Key Features

- Menu-driven interface
- Order placement functionality
- Order viewing functionality
- Order status update feature
- Support for multiple service types:
    - Wash
    - Iron
    - Wash + Iron
- Support for cloth types:
    - Normal
    - Woolen
- Dynamic pricing mechanism
- File-based data persistence

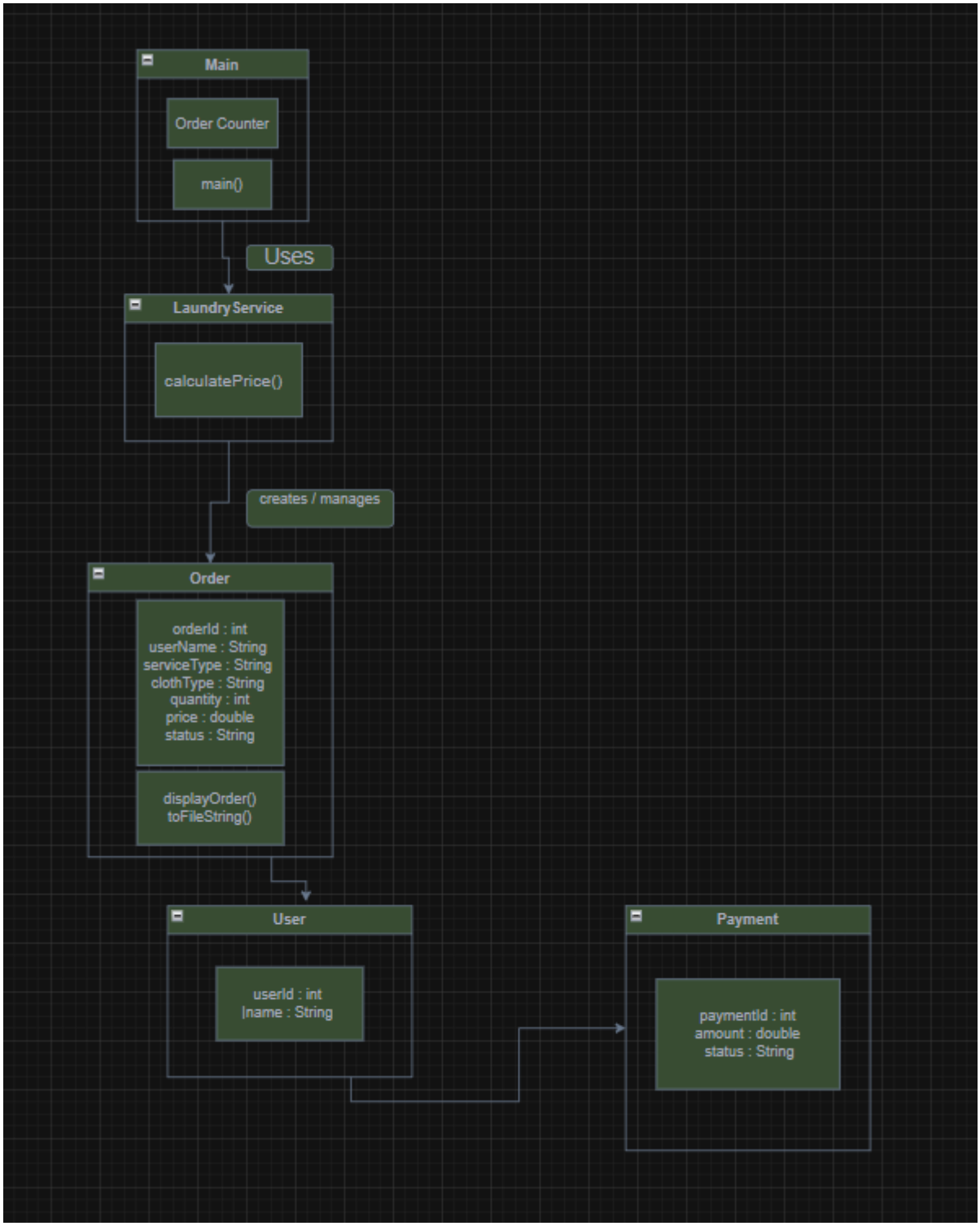
4. System Design

4.1 Use Case Diagram



Explanation:
This diagram shows the interaction between the user and the system. The user can place orders and view them, while the admin can update the status of orders and manage it.

4.2 Class Diagram



Explanation:

The class diagram represents the structure of the system. Key classes include:

- Main: Controls program execution
- Order: Stores order details
- LaundryService: Handles pricing logic
- User: Represents customer
- Payment: Represents transaction details

4.3 Flowchart

Explanation:
The flowchart shows the step-by-step process from program start to termination.

5. System Architecture

5.1 Overview

The system follows a **modular architecture**.

5.2 Layers

- **Presentation Layer:** Console interface
- **Business Logic Layer:** LaundryService class
- **Data Layer:** File handling (orders.txt)

This separation improves maintainability and scalability.

6. Implementation Details

6.1 Technologies Used

- Java (Core Java)

- Object-Oriented Programming Concepts

- File Handling (FileReader, FileWriter, BufferedReader)

- GitHub Codespaces

6.2 Modules Description

6.2.1 Main Module

- Handles user input
- Displays menu
- Controls flow of execution

6.2.2 Order Module

Stores:

- Order ID
- Customer Name
- Service Type
- Cloth Type
- Quantity
- Price
- Status

- Status

6.2.3 LaundryService Module

Implements pricing logic using conditional statements based on:

- Service type
- Cloth type

6.2.4 File Handling Module

- Saves orders into file
- Loads previous orders at startup
- Updates file when status changes

7. Pricing Mechanism

7.1 Pricing Table

The system uses a **dynamic pricing model**:

Service	Normal	Woolen
Wash	₹10	₹15
Iron	₹5	₹8
Wash + Iron	₹15	₹20

7.2 Explanation

Woolen clothes require special care and handling, resulting in higher pricing compared to normal clothes.

8. Algorithm / Working

8.1 Steps

1. Start the program
2. Load existing orders from file
3. Display menu options

4. Take user input
5. Perform selected operation:
    - Place Order
    - View Orders
    - Update Status
6. Save data to file
7. Repeat until exit

9. Output Screenshots

MENU:

```
No previous data found.
=== Smart Laundry Management System ===

1. Place Order
2. View Orders
3. Update Order Status
4. Exit
Choose option:
```

Order Placement:

```
=== Smart Laundry Management System ===

1. Place Order
2. View Orders
3. Update Order Status
4. Exit
Choose option: 1
Enter your name: muskan
Enter service (wash/iron/wash+iron):wash
Enter cloth type (normal/woolen): normal
Enter quantity: 4
Order placed successfully!

1. Place Order
2. View Orders
3. Update Order Status
4. Exit
Choose option:
```

Order Display:

```
1. Place Order
2. View Orders
3. Update Order Status
4. Exit
Choose option: 1
Enter your name: muskan
Enter service (wash/iron/wash+iron):wash
Enter cloth type (normal/woolen): normal
Enter quantity: 4
Order placed successfully!

1. Place Order
2. View Orders
3. Update Order Status
4. Exit
Choose option: 2

Order ID: 1
User: muskan
Service: wash
Quantity: 4
Price: 40.0
Status: Pending
clothtype: normal

1. Place Order
2. View Orders
3. Update Order Status
4. Exit
Choose option:
```

Status Update:

```
1. Place Order
2. View Orders
3. Update Order Status
4. Exit
Choose option: 3
Enter Order ID to update: 1
Enter new status (Pending/Washing/Completed): Completed
Order status updated!

1. Place Order
2. View Orders
3. Update Order Status
4. Exit
Choose option:
```

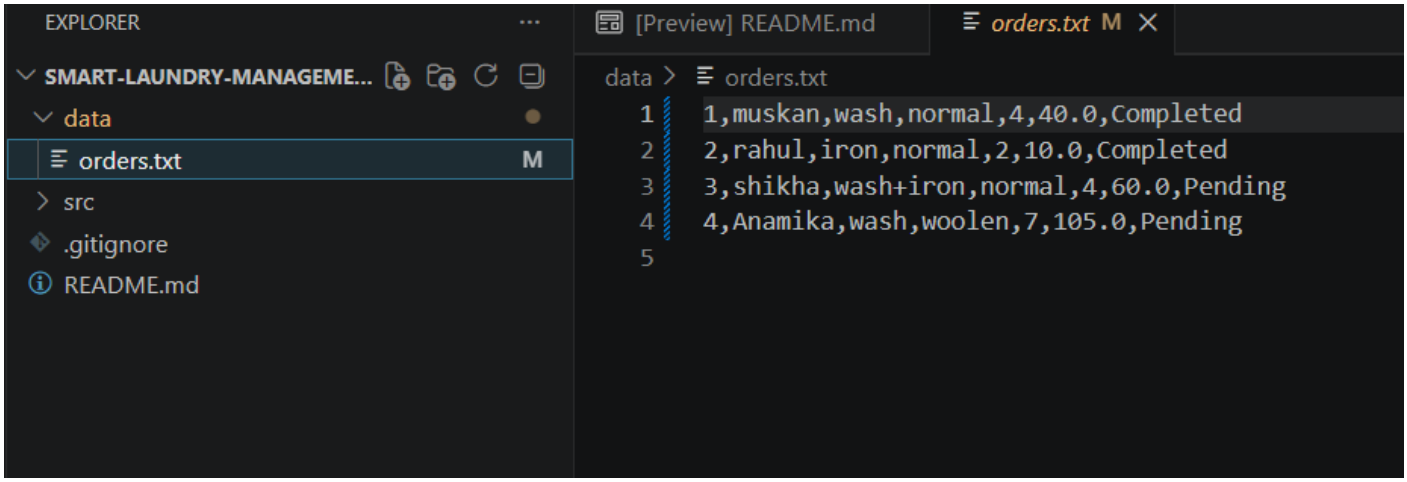
Exit:

```
1. Place Order
2. View Orders
3. Update Order Status
4. Exit
Choose option: 4
Thank you!
```

Include:

- Menu
- Order placement
- Order display
- Status update

Orders.txt → where all the orders will be stored and updated when the admin will update.



10. Testing

10.1 Test Scenarios

- Valid inputs
- Invalid service type
- File reading/writing
- Multiple order handling

-Result:

The system worked successfully for all tested scenarios.

10.2 Test Cases

Test Case 1: Place Order (Valid Input)

- **Test Case ID:** TC01
- **Objective:** To verify that a user can successfully place an order
- **Input:**
  - Name: Muskan
  - Service: wash
  - Cloth Type: normal
  - Quantity: 3
- **Expected Output:**

    - Order is created successfully
    - Price calculated correctly (₹10 × 3 = ₹30)
    - Data stored in orders.txt
- **Actual Output:** Order placed successfully
- **Status:** Pass

<u>Test Case 2</u>: Place Order with Wash + Iron

- **Test Case ID:** TC02
- **Objective:** To verify combined service pricing
- **Input:**
    - Name: Rahul
    - Service: wash+iron
    - Cloth Type: woolen
    - Quantity: 2
- **Expected Output:**
    - Price calculated correctly (₹20 × 2 = ₹40)
    - Order stored successfully
- **Actual Output:** Order placed successfully
- **Status:** Pass

<u>Test Case 3</u>: Invalid Service Input

- **Test Case ID:** TC03
- **Objective:** To verify system handles invalid service type
- **Input:**
    - Service: dryclean
- **Expected Output:**
    - Display error message: "Invalid service!"
    - Order should NOT be created
- **Actual Output:** Error displayed and order not saved

- **Status:** Pass

<u>Test Case 4</u>: View Orders (Non-empty List)

- **Test Case ID:** TC04
- **Objective:** To verify viewing of existing orders
- **Precondition:** At least one order exists
- **Input:** Choose option 2
- **Expected Output:**
    - All order details displayed correctly
- **Actual Output:** Orders displayed
- **Status:** Pass

<u>Test Case 5</u>: View Orders (Empty List)

- **Test Case ID:** TC05
- **Objective:** To verify behavior when no orders exist
- **Precondition:** orders.txt is empty
- **Input:** Choose option 2
- **Expected Output:**
    - Message: "No orders found!"
- **Actual Output:** Message displayed
- **Status:** Pass

<u>Test Case 6</u>: Update Order Status (Valid ID)

- **Test Case ID:** TC06
- **Objective:** To verify updating order status
- **Input:**
    - Order ID: 1
    - New Status: Washing
- **Expected Output:**

    - Status updated successfully
    - File updated with new status
- **Actual Output:** Status updated
- **Status:** Pass

<u>Test Case 7</u>: Update Order Status (Invalid ID)

- **Test Case ID:** TC07
- **Objective:** To verify system behavior for invalid order ID
- **Input:**
    - Order ID: 999
- **Expected Output:**
    - Message: "Order not found!"
- **Actual Output:** Message displayed
- **Status:** Pass

<u>Test Case 8</u>: File Persistence

- **Test Case ID:** TC08
- **Objective:** To verify data is stored permanently
- **Steps:**
    1. Place an order
    2. Exit program
    3. Restart program
- **Expected Output:**
    - Previously stored orders should be loaded
- **Actual Output:** Orders loaded successfully
- **Status:** Pass

<u>Test Case 9</u>: Multiple Orders Handling

- **Test Case ID:** TC09

- **Objective:** To verify system handles multiple orders
- **Input:** Place multiple orders
- **Expected Output:**
  - All orders stored and displayed correctly
- **Actual Output:** Works correctly
- **Status:** Pass

Test Case 10: Price Calculation Accuracy

- **Test Case ID:** TC10
- **Objective:** To verify correct price calculation
- **Input:**
  - Service: iron
  - Cloth Type: woolen
  - Quantity: 4
- **Expected Output:**
  - Price = ₹8 × 4 = ₹32
- **Actual Output:** Correct price displayed
- **Status:** Pass

11. Advantages

- Simple and user-friendly
- Reduces manual errors
- Efficient order tracking
- Persistent data storage
- Easy to maintain

12. Limitations

- No graphical interface

- No database integration
- Limited scalability
- No user authentication

13. Future Enhancements

- GUI using Java Swing
- Database integration (MySQL)
- Online payment system
- Mobile application
- Notification system
- Multi-user login system

14. Conclusion

- The Smart Laundry Management System successfully automates the process of managing laundry services in small environments. It improves efficiency, ensures accurate record-keeping, and provides a structured approach to handling orders.
- The project demonstrates the practical application of Java programming, object-oriented principles, and file handling techniques.